

SecurePDF - A Django-Based PDF Encryption and Decryption Web Application

Introduction

SecurePDF is a secure, user-friendly web application developed using Django that allows users to encrypt and decrypt PDF files with password protection. The application ensures user privacy, provides secure file handling, and supports automated file cleanup after processing. This tool is ideal for individuals or organizations looking to protect sensitive PDF documents during sharing or storage.

Objective

The core goal of this project was to:

- Build a web interface that allows users to upload a PDF file and apply password-based encryption.
- Provide functionality to decrypt PDF files using the correct password.
- Ensure encrypted/decrypted files are automatically deleted after use to maintain system hygiene.
- Deploy the solution on a live cloud platform for public access.

Tech Stack

- **Backend Framework:** Django (Python)
- **Frontend:** HTML5, CSS3, Js
- **PDF Library:** PyPDF2
- **Deployment:** Render (render.com)
- **Others:** Gunicorn, Whitenoise, FileSystemStorage, UUID, OS, datetime

Key Features

- Password-based encryption of uploaded PDFs.
- Password-based decryption with error handling.
- Timestamped file naming convention to prevent collisions.
- Auto-cleanup of temporary uploaded files.
- Session-based file access for security.
- User feedback via Django message framework.
- Deployed and publicly accessible at: <https://securepdf-np2x.onrender.com>

Function Flow

1. PDF Encryption

- a. User navigates to /encrypt/ page.
- b. Uploads a PDF file and provides a password.
- c. The backend reads and encrypts the PDF using PyPDF2.
- d. The encrypted file is saved temporarily and returned to the user for download.
- e. Original uploaded file is deleted from the server.

2. PDF Decryption

- a. User navigates to /decrypt/ page.
- b. Uploads a previously encrypted PDF and enters the password.
- c. Backend decrypts the file and returns the decrypted version for download.
- d. Original file is removed post-decryption.

Security & Optimization

- All files are temporarily stored in `uploaded_pdfs/` directory and deleted post-processing.
- Encrypted and decrypted files are stored with unique names using UUID and timestamp.
- Only valid PDFs are accepted through form validation.
- Files are never exposed via direct filesystem paths, instead Django serves through `MEDIA_URL`.

Deployment Configuration

- `Render.com` used for seamless CI/CD integration.
- `gunicorn` and `whitenoise` configured for production-ready deployment.
- Static and media files are served correctly via Django settings.

Snapshots

```
12 def encrypt_pdf(input_file_path, password):
13     try:
14         reader = PdfReader(input_file_path)
15         writer = PdfWriter()
16
17         for page in reader.pages:
18             writer.add_page(page)
19
20         writer.encrypt(password)
21
22         output_filename = generate_unique_filename("encrypted_output")
23         output_path = os.path.join(settings.MEDIA_ROOT, output_filename)
24         with open(output_path, "wb") as f:
25             writer.write(f)
26
27         return output_path
28     except Exception as e:
29         raise Exception(f"Encryption failed: {e}")
```



```
31 def decrypt_pdf(input_file_path, password):
32     try:
33         reader = PdfReader(input_file_path)
34
35         if reader.is_encrypted:
36             if not reader.decrypt(password):
37                 raise Exception("Wrong password or decryption failed.")
38             else:
39                 raise Exception("The file is not encrypted.")
40
41         writer = PdfWriter()
42         for page in reader.pages:
43             writer.add_page(page)
44
45         output_filename = generate_unique_filename("decrypted_output")
46         output_path = os.path.join(settings.MEDIA_ROOT, output_filename)
47         with open(output_path, "wb") as f:
48             writer.write(f)
49
50         return output_path
51     except Exception as e:
52         raise Exception(f"Decryption failed: {e}")
```



Challenges Faced

- Handling encrypted PDF decryption errors.
- Managing temporary file cleanup after processing.
- Ensuring deployment compatibility with `Render's` ephemeral storage.

Future Improvements

1. Support for bulk PDF uploads.
2. UI enhancement for better user experience.
3. QR code generation for secure PDF download links.

Conclusion

SecurePDF is a compact, reliable solution to protect PDF documents online. It combines essential security with an easy-to-use interface, making it useful for students, professionals, and businesses.